SYSTEM CALLS

ABOUT SYSTEM CALLS:

A system call is a request from a user program to the OS kernel to perform a specific function.

->A system call is an interface between a user program and the operating system kernel that allows the program to request OS services.

->A system Call acts as a bridge between a user program and the OS kernel.

->CPU will be executed user application in user space.

->OS executed in the kernel space.

->CPU having 2 modes.

1. user mode
2. kernel mode.

->All the user application will be started to executed in the user space.

->when the application needed hard ware it will switch into the kernel space.

TYPES OF SYSTEM CALLS:

Process control: fork(), exec(), wait(), exit()

File management: open(), read(), write(), close()

Device management: ioctl()

Communication: pipe(), socket(),SHM();

Information management: getpid()

HOW DOES A SYSTEM CALL WORKS?

User program → System call → OS kernel → Result returned to the program.

Example: When a programcalls write(), it requests the kernel to write data to a file or terminal.

why do we need to study system calls?

We study systems calls to understand how the programs communicate with the OS kernel and use its services.

File handling: To open, read, write, and close files using open(), read(), write(), and close().

Process management: To create and control processes using fork(), exec(), and wait()**.**

Memory management: To allocate, map, and release memory using calls such as mmap() and munmap().

Inter-process communication (IPC): To exchange data between processes using pipe() and shmget() or related shared-memory interfaces.

OS understanding: To understand how applications request services from the kernel.

HOW TO USE SYSTEM CALLS IN C?

system calls can be called from a c program using the appropriate header files.

eg:

```
#include <unistd.h>
int main(void)
{
   write(1, "Hello\n", 6);
   return 0;
}
```

what is wrapper routine?

A wrapper routine is a libary fuction that provides a convenient way to invoke an underlying system call.

for example:

write(1,"hello\n",6);

The write () function provided by the C library typically acts as a wrapper around the Linux write system call.

The kernel executes the requested operation and returns a result. When a system call fails, the C library wrapper commonly returns -1 and sets err no to indicate the error.

NOTE: not every library function is a system call wrapper. For example, printf() is a C library function that formats output and may use write () internally when its output is flushed.

LINUX SYSTEM CALLS: OPEN() AND CLOSE()

1.OPEN()SYSTEM CALL

open() requests the Linux kernel to open a file and returns a file descriptor that the process can use to access it.

SYNTAX:

int open(const char *pathname, int flags);

NOTE: To create a file if it does not exist, use the three-argument form:

int open(const char *pathname, int flags, mode_t mode);unistd.h

HEADER FILES:

#include<fcntl.h>

#include<unistd.h>

Common flags:

Flag	**Purpose**
O_RDONLY	Open for reading
O_WRONLY	Open for writing
O_RDWR	Open for reading and writing

O_CREAT Create the file if it does not exist

O_TRUNC Truncate an existing regular file when opened for writing

O_APPEND Position each write at the end of the file

Flags can be combined using the bitwise OR operator (|).

2.CLOSE ()system call

close() requests the kernel to close a file descriptor, releasing that descriptor for reuse.

SYNTAX:

int close (int fd);

NOTE: Closing an FD does not necessarily delete the file. It closes that process's reference to the open resource.

Return values

- Success: returns 0.

- Failure: returns -1 and sets err no.

Read () SYSTEM CALL

Read () requests the kernel to read bytes from a file descriptor and copy them into a buffer in your program's memory.

SYNTAX:

ssize_t read(int fd, void *buf, size_t count);

HEADER FILE:

#include <unistd.h>

2.write () system call

Write () requests the kernel to write bytes from your program's memory to a file descriptor.

syntax:

ssize_t write(int fd, const void *buf, size_t count);

internal structure of file handling in Linux:

1. File descriptor table

2. Open-file description

3. Inode.

WHAT IS FILE DESCRIPTOR (FD):

A file descriptor (FD) is a non-negative integer used by a process to identify an open file or another input/output resource.

WHO CREATE THE FD?

->OS will created the FD table for every process.

->OS will open the FD files by default.

->When a program calls open (), the kernel opens the file and returns an available FD.

->The kernel normally allocates the lowest available FD number.

STANDARD FILE DESCRIPTORS?

Every process normally starts with three standard file descriptors:

FD	Name	Purpose
0	**Standard input (stdin)**	**Receives input**
1	**Standard output (stdout)**	**Displays normal output**
2	**Standard error (stderr)**	**Displays error messages**

NOTE: BY USING THIS COMMAND WE CAN OBSERVE FD IN LINUX ls -l /proc/self/fd

NOTE: ulimit -n shows the shell's open-file limit. A process can have a different limit, and other system constraints may apply.

2.OPEN-FILE DESCRIPTIOR:

An open-file description represents an open instance of a file. In Linux, this is implemented using a kernel structure called struct file

It maintains information such as:

- File offset: The current position for reading or writing.

- File status flags: Such as O_APPEND and O_NONBLOCK.

- File reference: The associated file object and filesystem operations.

Example: How the file offset changes

Suppose data.txt contains:

HELLO LINUX

Your program opens it and reads five bytes:

char buffer[6];

int fd = open("data.txt", O_RDONLY);

read(fd, buffer, 5);

H E L L O SP L I N U X

offset=0

H E L L O SP L I N U X

offset=5

The next read through the same open-file description will normally start at offset 5, so it reads from the space after HELLO.

This is why the open-file description is important: it remembers the current file position.

What happens if two descriptors refer to the same open-file description?

For example:

int fd1 = open ("data.txt", O_RDONLY);

int fd2 = dup(fd1);

Now fd1 and fd2 refer to the same open-file description.

If you read five bytes using fd1, the shared file offset advances by five. A subsequent read using fd2 will normally continue from that new position.

This is a key concept for understanding dup() and dup2().

4.inode:

An inode is a filesystem data structure that stores metadata about a file. Filesystems such as ext4 use inodes to represent files and directories.

An inode typically stores:

- File type

- File permissions

- Owner and group IDs

- File size

- Timestamps

- References to the file's data blocks

- Link count

Filename	**Inode number**
data.txt	1201
notes.txt	1202
image.jpg	1203

NOW WE STUDY ABOUT THE lseek (), stat (), and fstst ()

1.lseek ()->change the file offset

lseek () changes the current position in an open file. This position is maintained by the open-file description.

syntax:

off_t lseek(int fd, off_t offset, int whence);

fd: file descriptor.

offset: number of bytes to move relative to the selected reference point.

whence: the reference point.

whenceMeaning

SEEK_SET Beginning of the file

SEEK_CUR Current position

SEEK_END End of the file

Example

Suppose data.txt contains:

ABCDEFGHIJ

program:

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
 int main(void)
{
   char ch;
   int fd = open("data.txt", O_RDONLY);

   if (fd == -1)
   {
      perror("open");
      return 1;
   }

   lseek(fd, 4, SEEK_SET);

   if (read(fd, &ch, 1) == 1)
      printf("%c\n", ch);

   close(fd);
   return 0;
}
```

output: E

2.stat ()->get file information

Stat () retrieves metadata about a file identified by its pathname.

syntax:

int stat (const char *pathname, struct stat *statbuf);

It can provide:

- File size
- Permissions
- Inode number
- File type
- Owner and group IDs
- Timestamps

3.fstat ()->get file information using an FD

stat and fstat is both are similar working fstat()retrieves metadata through an already-open file descriptor.

SYNTAX:

int fstat(int fd, struct stat *statbuf);

program

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
int main(void)
{
   struct stat info;
int fd = open("data.txt", O_RDONLY);
if (fd == -1)
   {
      perror("open");
      return 1;
   }

   if (fstat(fd, &info) == -1)
   {
      perror("fstat");
      close(fd);
```

```
        return 1;
    }

    printf("Size: %lld bytes\n", (long long)info.st_size);

    close(fd);
    return 0;
}
```

Difference between stat () and fstat ()

stat()	**fstat()**
Uses a pathname	Uses an open FD
Locates the file by name	Examines the object referenced by the FD
Does not require you to open the file first	Requires a valid open FD